

UNIVERSIDAD TECNOLÓGICA NACIONAL
TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

PROGRAMACIÓN I

TRABAJO PRÁCTICO INTEGRADOR

**“Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas”**

INTEGRANTES:

Salcedo Scardino Julián
Comisión: M26 C1-16

Suarez Taboada Mateo Ezequiel
Comisión M26 C1-03

PROFESORES:

Ariel Enferrel · Martín A. García · Cinthia Rigoni

Rosario, Santa Fe — Argentina
15 de junio de 2026

Índice

Introducción	2
1. Marco Teórico	2
1.1 Listas	2
1.2 Diccionarios	2
1.3 Funciones	3
1.4 Estructuras Condicionales	3
1.5 Estructuras Repetitivas.....	3
1.6 Ordenamientos	4
1.7 Estadísticas Básicas	4
1.8 Archivos CSV	4
2. Objetivo del Trabajo	5
3. Decisiones Técnicas y Arquitectura.....	5
3.1 Cómo organizamos el proyecto	5
3.2 Por qué usamos listas de diccionarios.....	5
3.3 Flujo de ejecución del sistema	6
3.4 Validaciones implementadas.....	7
4. Cómo lo desarrollamos	7
5. Qué logramos.....	8
6. Lo que nos costó y lo que aprendimos	8
6.1 Dificultades.....	8
6.2 Conclusiones	9
7. Repositorio y video	9
8. Bibliografía / Webgrafía	9

Introducción

Para este trabajo teníamos que construir un sistema en Python que manejara información sobre países: cargar datos desde un archivo CSV, mostrar un menú y permitir buscar, filtrar, ordenar y sacar estadísticas. En principio parece sencillo, pero a medida que fuimos avanzando nos dimos cuenta de que había que pensar bien cómo organizarlo para que no se convirtiera en un caos.

Lo primero que definimos fue separar el código en módulos. Cada archivo tiene una sola responsabilidad: uno carga el CSV, otro maneja las búsquedas, otro los filtros, y así. Eso nos facilitó muchísimo el desarrollo porque en todo momento sabíamos exactamente dónde tocar cuando algo no andaba.

El programa corre en consola, no tiene interfaz gráfica. Al iniciarse lee el archivo `países.csv`, arma la lista de países en memoria y muestra el menú. Cualquier cambio que haga el usuario se guarda automáticamente al salir, o en el momento en que agrega, modifica o elimina un registro.

1. Marco Teórico

1.1 Listas

Una lista en Python es básicamente una colección de cosas en orden. Se puede agregar, quitar, recorrer y ordenar. Lo que la hace especial frente a otros lenguajes es que puede guardar cualquier tipo de dato, incluso otras listas o diccionarios, y eso la hace muy flexible.

En nuestro sistema, la variable `países` es una lista que contiene todos los países cargados desde el CSV. Todo lo demás —buscar, filtrar, ordenar— trabaja sobre esa lista. Si no existiera esa estructura central, cada operación tendría que ir a leer el archivo de vuelta, lo cual sería mucho más lento e incómodo.

```
# La lista principal del sistema
países = []

# Agregar un país nuevo
países.append({'nombre': 'Argentina', 'población': 45376763, ...})

# Recorrer todos los países
for pais in países:
    print (pais['nombre'])
```

1.2 Diccionarios

Un diccionario guarda datos en pares clave-valor. En vez de acceder a los datos por posición (`pais[0]`, `pais[1]`...), los accedes por nombre (`pais['nombre']`, `pais['poblacion']`). Eso hace que el código sea mucho más fácil de leer, especialmente cuando hay cuatro o cinco campos por registro.

Cada país de nuestro sistema es un diccionario. Esa decisión la tomamos al principio y funcionó bien: en ningún momento hubo confusión sobre qué dato era cada cosa, porque los nombres de las claves son descriptivos.

```
# Así se ve un país en memoria
pais = {
    'nombre':      'Argentina',
    'poblacion':   45376763,
    'superficie':  2780400,
    'continente':  'América'
}

# Acceder a un campo es directo
print (pais['nombre']) # Salida: Argentina
```

1.3 Funciones

Una función es un bloque de código que hace una cosa y tiene un nombre. La idea es que, si necesitas hacer lo mismo en varios lugares, lo escribís una sola vez y lo llamas cuando lo necesitas. Pero más allá de reutilizar código, las funciones sirven para dividir el problema en partes manejables.

En nuestro proyecto aplicamos el principio de que cada función hace una sola cosa. Hay una función para agregar un país, otra para buscarlo, otra para ordenar, etc. Eso hizo que cuando algo fallaba pudiéramos encontrar el problema rápido, sin tener que revisar cientos de líneas de código mezclado.

```
def buscar_pais(paises):
    """Busca paises por coincidencia parcial o exacta en el nombre."""
    termino = input('ingresa el nombre o parte del nombre: ').strip()
    resultados = [p for p in paises
                   if termino.lower() in p['nombre'].lower()]
    return resultados
```

1.4 Estructuras Condicionales

Los condicionales (if, elif, else) le permiten al programa tomar decisiones. Sin ellos, el código haría siempre lo mismo sin importar qué ingrese el usuario. En la práctica, los usamos en casi todos lados: para validar datos, para ver si un país ya existe, para controlar qué opción eligió el usuario.

Un caso concreto es cuando el usuario quiere agregar un país: primero chequeamos que el nombre no esté vacío, después vamos si ya existe otro igual, y recién entonces lo agregamos. Si no hubiera condicionales, cualquier dato pasaría sin problemas y el sistema quedaría en un estado inconsistente.

```
if not nombre:
    print('ERROR] El nombre no puede estar vacio.')
elif buscar_exacto(paises, nombre) is not None:
    print('[ERROR] Ya existe un pais con ese nombre,')
else:
    paises.append(nuevo_pais)
    print('[OK] Pais agregado correctamente.')
```

1.5 Estructuras Repetitivas

Los bucles permiten repetir código sin tener que escribirlo varias veces. Python tiene dos: for, que recorre una colección elemento por elemento, y while, que sigue ejecutándose mientras una condición sea verdadera. Los dos aparecen en nuestro proyecto constantemente.

El menú principal es un while True que sigue mostrando opciones hasta que el usuario escribe 0 para salir. Los recorridos de la lista de países para buscar, filtrar o calcular estadísticas usan for. Sin estos bucles, el programa no podría funcionar de forma interactiva ni procesar más de un país a la vez.

```
# El menú sigue activo hasta que el usuario elige '0'
while True:
    opcion = input ('Elegí una opcion: ').strip()
    if opción == '0':
        guardar_paises(paises), ARCHIVO_CSV
        break

# Recorrido para encontrar coincidencias
for pais in paises:
    if termino.lower() in pais ['nombre'].lower():
        resultados.append(pais)
```

1.6 Ordenamientos

Python tiene la función `sorted()` que devuelve una lista ordenada sin tocar la original. Con el parámetro `key` le decís por qué criterio ordenar, y con `reverse` si querés ascendente o descendente. Es una de las funciones más cómodas del lenguaje porque evita tener que implementar el algoritmo de ordenamiento a mano.

En nuestro sistema usamos `sorted()` para los tres criterios: nombre (comparación de texto), población y superficie (comparación numérica). El usuario elige el criterio y la dirección, y el resultado se muestra en pantalla sin modificar la lista original.

```
# Ordenar por población de mayor a menor
Ordenados = sorted(paises, key=lambda p: p['poblacion'], reverse=True)

# Ordenar por nombre de A a Z
ordenados = sorted(paises, key=lambda p: p['nombre'])
```

1.7 Estadísticas Básicas

Para las estadísticas usamos las funciones nativas de Python: `max()` y `min()` para encontrar el mayor y el menor, y `sum()` para la suma total. Combinadas con una expresión generadora, el cálculo del promedio queda en una sola línea. No necesitamos ninguna librería externa para esto.

Para contar países por continente usamos un diccionario de conteo: recorreremos la lista con un `for` y por cada país sumamos 1 al continente correspondiente. Si el continente todavía no está en el diccionario, lo iniciamos en cero con `.get()`. Es una técnica sencilla pero muy útil.

```
mayor = max(paises, key=lambda p: p['poblacion'])
menor = min(paises, key=lambda p: p['poblacion'])
promedio = sum(p['poblacion'] for p in paises) / len(paises)

# Conteo de países por continente
conteo = {}
for pais in paises:
    cont = pais['continente']
    conteo[cont] = conteo.get(cont, 0) + 1
```

1.8 Archivos CSV

Un CSV es un archivo de texto donde cada línea es un registro y los campos están separados por comas. Es el formato más simple para guardar datos tabulares y tiene la ventaja de que cualquier programa puede leerlo, desde Python hasta Excel. No requiere ninguna librería especial.

Python incluye el módulo `csv` en su biblioteca estándar. Nosotros usamos `DictReader` para leer, que convierte cada fila en un diccionario usando los encabezados como claves, y `DictWriter` para escribir de vuelta. Eso mantiene consistencia entre cómo leemos y cómo guardamos los datos.

```
Import csv

# Leer: cada fila del CSV se convierte en un diccionario
with open('paises.csv', encoding='utf-8') as archivo:
    lector = csv.DictReader(archivo)
    for fila in lector:
        paises.append(fila)

# Escribir: cada diccionario se convierte en una fila del CSV
with open('paises.csv', 'w', encoding='utf-8', newline='') as archivo:
    escritor = csv.DictWriter(archivo, fieldnames=CAMPOS)
    escritor.writeheader()
    escritor.writerows(paises)
```

2. Objetivo del Trabajo

El objetivo general es desarrollar una aplicación funcional en Python 3 que gestione un dataset de países, integrando todos los conceptos trabajados en Programación 1: listas, diccionarios, funciones, estructuras de control, archivos CSV, ordenamientos y estadísticas.

Más allá del programa en sí, lo que se busca es demostrar que sabemos analizar un problema, dividirlo en partes, implementar cada parte de forma limpia y documentar el resultado. En concreto, el sistema tiene que poder:

- Leer y escribir un archivo CSV con datos de países.
- Permitir agregar, actualizar, buscar y eliminar registros desde un menú interactivo.
- Filtrar el dataset por continente, rango de población y rango de superficie.
- Ordenar los países por nombre, población o superficie de forma ascendente o descendente.
- Mostrar estadísticas del dataset: máximos, mínimos, promedios y conteo por continente.
- Validar todas las entradas del usuario y mostrar mensajes de error claros.

3. Decisiones Técnicas y Arquitectura

3.1 Cómo organizamos el proyecto

Desde el principio decidimos dividir el código en archivos separados, uno por funcionalidad. No porque fuera obligatorio, sino porque aprendimos que meter todo en un solo archivo hace que sea muy difícil encontrar errores y hacer cambios sin romper otra cosa.

Archivo	Qué hace
main.py	Es el punto de entrada. Muestra el menú y coordina el resto de los módulos.
carga.py	Lee el CSV al iniciar y lo sobrescribe cuando hay cambios. También valida el formato.
operaciones.py	Agregar, actualizar, buscar y eliminar países. Acá están todas las validaciones de entrada.
filtros.py	Filtra por continente, rango de población y rango de superficie.
ordenamiento.py	Ordena el dataset por nombre, población o superficie, ascendente o descendente.
estadísticas.py	Ordena el dataset por nombre, población o superficie, ascendente o descendente.
países.csv	El dataset base con 24 países reales de todos los continentes.

3.2 Por qué usamos listas de diccionarios

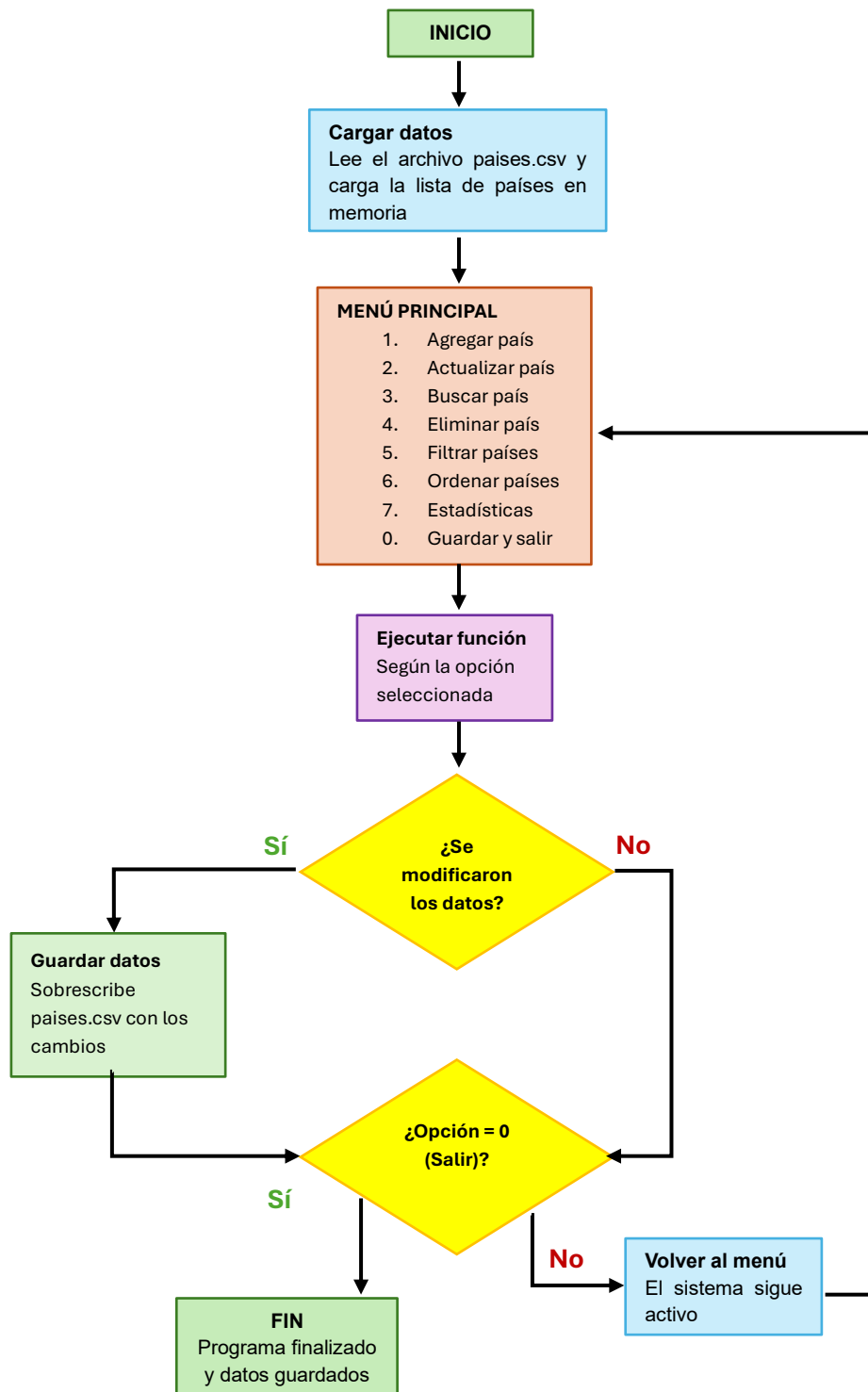
Podríamos haber guardado los datos de otras formas, por ejemplo, en listas de listas. Pero elegimos listas de diccionarios porque hace el código mucho más claro. Cuando escribís `pais['nombre']` entendés al instante qué estás accediendo. Si escribís `pais[0]` tenés que acordarte de que la posición 0 es el nombre, lo cual es innecesariamente confuso.

```
# Cada país en memoria se ve así
Pais = {
    'nombre': str,      # Ej: 'Argentina'
    'poblacion': int,    # Ej: 45376763
    'superficie': int,   # Ej: 2780400 (en km²)
    'continente': str    # Ej: 'América'
}

# La lista completa
países= [pais1, pais2, pais3, ...]
```

3.3 Flujo de ejecución del sistema

Este esquema muestra cómo corre el programa desde que lo inicias hasta que salís:



3.4 Validaciones implementadas

Uno de los aprendizajes del proyecto fue que sin buenas validaciones el sistema se rompe de formas que no esperas. Estos son los controles que implementamos:

- Campos vacíos: si el nombre o el continente quedan en blanco, el sistema lo rechaza y vuelve a pedir el dato.
- Tipos de datos: población y superficie tienen que ser números enteros mayores a cero. Si no, se muestra el error y se pide de nuevo.
- Duplicados: antes de agregar un país se cheque que no exista otro con el mismo nombre. La comparación ignora mayúsculas y minúsculas.
- Rangos: en los filtros numéricos, el mínimo no puede ser mayor que el máximo.
- Errores en el CSV: si una fila tiene datos mal formateados, se la ignora con un aviso y se sigue cargando el resto.
- Búsquedas sin resultados: en lugar de mostrar una lista vacía sin explicación, se muestra un mensaje claro.

4. Cómo lo desarrollamos

No arrancamos escribiendo código directamente. Primero analizamos la consigna, identificamos todo lo que tenía que hacer el sistema y definimos los módulos antes de escribir una sola línea. Eso nos ahorró tiempo después porque no tuvimos que reorganizar el código a mitad de camino.

Fuimos construyendo de a módulos: primero la carga del CSV, después las operaciones básicas, después los filtros, el ordenamiento y las estadísticas, y por último el menú principal que los conecta. Cada módulo lo probamos por separado antes de integrarlo.

Etapas	Qué hicimos
1	Leímos la consigna completa y anotamos todo lo que tenía que hacer el sistema.
2	Definimos los módulos y qué responsabilidad tendría cada uno.
3	Creamos el archivo paises.csv con 24 países reales de todos los continentes.
4	Implementamos carga.py: lectura, escritura y validación del CSV.
5	Implementamos operaciones.py: agregar, actualizar, buscar y eliminar países.
6	Implementamos filtros.py, ordenamiento.py y estadísticas.py
7	Conectamos todo en main.py con el menú principal.
8	Probamos cada funcionalidad, corregimos errores y ajustamos detalles.
9	Subimos el proyecto a GitHub y escribimos el README y este informe.

La división por módulos también nos facilitó trabajar en equipo: cada uno podía avanzar en su parte sin pisar el trabajo del otro. Después revisamos el código del compañero antes de integrar, lo que ayudó a detectar errores y mantener un estilo consistente.

5. Qué logramos

El sistema cumple con todo lo que pedía la consigna. Además, agregamos la opción de eliminar países, que no estaba en los requisitos, pero nos pareció necesaria para que el sistema fuera completo. La tabla siguiente resume el estado de cada funcionalidad:

Funcionalidad	Estado	Módulo
Agregar país con validación completa	✓ Completo	operaciones.py
Actualizar población y superficie	✓ Completo	operaciones.py
Buscar por nombre (parcial o exacto)	✓ Completo	operaciones.py
Eliminar país con confirmación (extra)	✓ Completo	operaciones.py
Filtrar por continente	✓ Completo	filtros.py
Filtrar por rango de población	✓ Completo	filtros.py
Filtrar por rango de superficie	✓ Completo	filtros.py
Ordenar por nombre, población y superficie (asc/desc)	✓ Completo	ordenamiento.py
Estadísticas de población: máx, mín y promedio	✓ Completo	estadísticas.py
Estadísticas de superficie: máx, mín y promedio	✓ Completo	estadísticas.py
Cantidad de países por continente	✓ Completo	estadísticas.py
Lectura y escritura del CSV	✓ Completo	carga.py
Validaciones y mensajes de error en todos los módulos	✓ Completo	Todos

6. Lo que nos costó y lo que aprendimos

6.1 Dificultades

Hubo varios momentos en los que nos trabamos y tuvimos que buscar cómo resolverlo:

- El tema de los encodings nos tomó por sorpresa. La primera vez que corrimos el programa con países que tienen tildes en el nombre, los caracteres salían mal. La solución fue simple (agregar `encoding='utf-8'` al abrir el archivo), pero primero tuvimos que entender por qué pasaba.
- Las validaciones nos dieron más trabajo del que esperábamos. Cada vez que pensábamos que estaban completas, encontrábamos un caso que no habíamos contemplado: ¿qué pasa si el usuario escribe un espacio en blanco? ¿y si escribe letras donde va un número? Terminamos revisando esas funciones varias veces.
- Definir bien los límites entre módulos también generó dudas al principio. Había funciones auxiliares que usaban varios módulos y no estaba claro dónde ponerlas. Lo resolvimos dejándolas en el módulo que más las necesitaba, definidas como funciones internas con el prefijo `_`.
- Las comparaciones de texto fueron otro punto a revisar: buscar 'argentina' y 'Argentina' tiene que dar el mismo resultado. Tuvimos que asegurarnos de aplicar `.lower()` en todos los lugares donde se comparan nombres.

6.2 Conclusiones

Este trabajo nos hizo ver en la práctica algo que en clase es difícil de entender completamente: cómo los distintos conceptos que estudiamos se conectan para construir algo real. Las listas, los diccionarios, las funciones, los archivos... cada uno por separado parece un tema de clase, pero al juntarlos en un sistema empezás a entender para qué sirve cada uno.

Lo que más nos sirvió fue modularizar desde el principio. Hubo momentos en que encontramos bugs y los pudimos ubicar rápido porque sabíamos exactamente en qué módulo tenía que estar el problema. Si hubiéramos metido todo en un solo archivo, eso hubiera sido mucho más complicado.

También aprendimos que las validaciones no son un detalle de último momento. Un sistema sin validaciones falla de maneras inesperadas y difíciles de rastrear. Dedicarles tiempo al principio nos ahorró problemas más adelante.

En cuanto al trabajo en equipo, dividir el proyecto en módulos fue la clave para poder avanzar sin bloquearnos. Revisar el código del otro antes de integrarlo fue algo que al principio parecía innecesario pero que terminó siendo muy útil para encontrar errores que uno solo no ve.

7. Repositorio y video

Todo el código, el CSV y este documento están disponibles en el repositorio de GitHub:

Repositorio de GitHub: https://github.com/BrUHmaN-43/tpi_paises

— Video demostración: <https://youtu.be/xqEgS6bt7fA>

El README del repositorio tiene instrucciones de instalación y ejecución, ejemplos de uso con entrada y salida esperada, datos del equipo docente e integrantes, y los links al video y a este PDF.

8. Bibliografía / Webgrafía

- Python Software Foundation. (2024). Python 3 Documentation. <https://docs.python.org/3/>
- Python Software Foundation. (2024). csv — CSV File Reading and Writing. <https://docs.python.org/3/library/csv.html>
- Python Software Foundation. (2024). Built-in Functions (sorted, max, min, sum). <https://docs.python.org/3/library/functions.html>
- Python Software Foundation. (2024). Data Structures — Lists y Dictionaries. <https://docs.python.org/3/tutorial/datastructures.html>
- Real Python. (2024). Reading and Writing CSV Files in Python. <https://realpython.com/python-csv/>
- Real Python. (2024). How to Use sorted() and sort() in Python. <https://realpython.com/python-sort/>
- Real Python. (2024). Dictionaries in Python. <https://realpython.com/python-dicts/>
- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media. <https://www.oreilly.com/library/view/learning-python-5th/9781449355722/>